Autómata del Boletín 01, ejercicio 1: suma, resta, producto, cociente y resto

Caso práctico

Es el primer ejercicio del primer boletín y el que se usó para presentar la técnica: dos números, dos botones y unas pocas páginas distintas. **Carlos** lo ha programado siguiendo el autómata del material y funciona... hasta que **María** escribe un 0 como segundo número y la página termina con un error fatal. —El autómata no estaba mal dibujado —le dice—: le faltaba una condición. Y eso es justo lo que la comprobación de predicados sirve para encontrar.

Este documento rehace el autómata de los ejemplos del repositorio para el Boletín 01-E01 (no incluido en este material) con el formato de los demás ejemplos y lo completa. Se **conserva la numeración** del original (PL0, PL1, AC0–AC3, ES0–ES2): lo nuevo va a continuación, para poder comparar las dos versiones.

1.- Enunciado

Desarrolla un script que solicite que el usuario introduzca 2 números, los obtenga y produzca como salida la suma, resta, producto, cociente y resto de ambos números. Ejemplo del boletín:

En la página	Valor
Introduzca el primer número	30
Introduzca el segundo número	10
La suma es	40
La resta es	20
El producto es	800
El resto es	0

Debes conocer

El ejemplo del boletín tiene dos erratas: el producto de 30 y 10 es **300**, no 800; y el enunciado pide también el **cociente**, que no aparece en el ejemplo (ni en la solución del material). Aquí se calcula como cociente **entero** (`intdiv`), que es el que acompaña al resto: 30 entre 10 da cociente 3 y resto 0; −7 entre 2, cociente −3 y resto −1.

2.- Análisis

- **Eventos entrantes.** Los dos botones son `submit` con nombre (`botonEnviar`, `botonReset`): *Enviar* y *Reset*. La primera carga, sin ningún botón, se trata como *Reset*, igual que en la solución del material.

- **Estados.** Dos: arranque (ST0) y atención de peticiones (ST1). Tras cada respuesta la página vuelve a esperar dos números.
- **Predicados.** Los dos del original (cada número es un entero válido) y uno que faltaba: **el segundo número no es 0** (PL2). Sin él, el camino de los resultados calcula `$numero1 % $numero2` con un 0 y PHP 8 lanza `DivisionByZeroError`. La tabla de descriptores del original ya tenía el hueco PL2 sin rellenar.
- **Un camino nuevo.** Con el segundo número 0 sí se pueden dar la suma, la resta y el producto: se muestran, con un aviso de que el cociente y el resto no se pueden calcular (ES3).
- **Acciones previas.** Leer y validar los dos números (AC5, AC6). En el original se hacía con `formatoEnteroValido()` y el texto `"NOVALIDO"`; aquí, con `FILTER_VALIDATE_INT` y un rango (ver el apartado 5).

3.- Diagrama de estado-transición ampliado

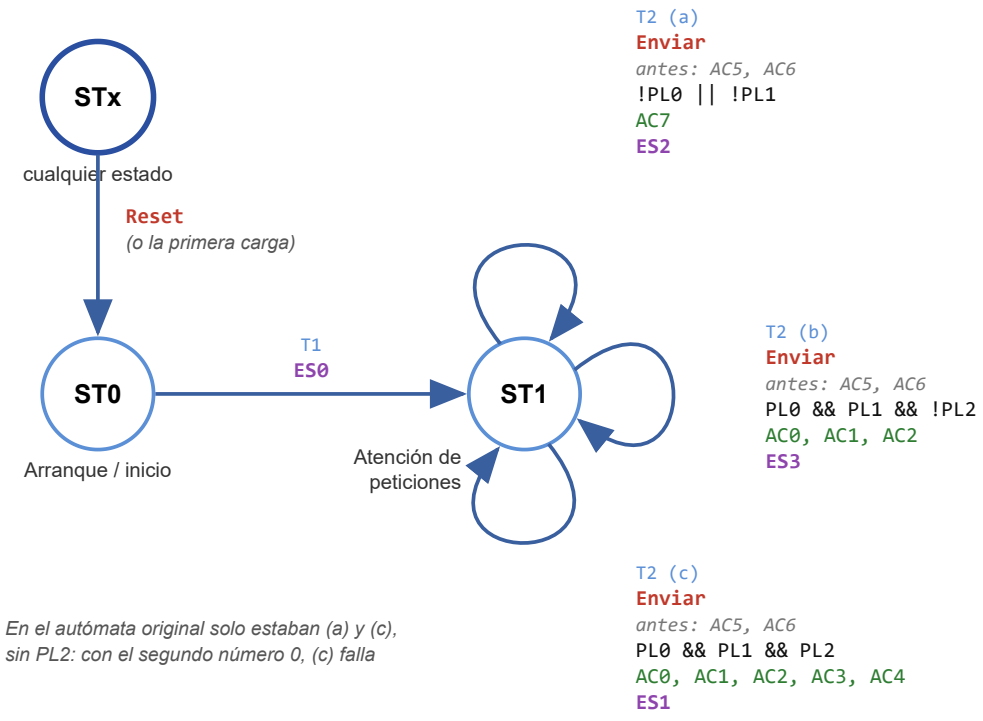


Figura 1. Diagrama del Boletín 01, ejercicio 1 (completado con PL2 y el camino (b))

4.- Autómata

4.1.- Tabla de descriptores

Tipo	Abreviatura	Nombre	Significado
Evento entrante	EE0	Enviar	se pulsa el botón "Enviar"
Evento entrante	EE1	Reset	se pulsa el botón "Reset" (o primera carga, sin botón)
Evento saliente	ES0	formularioInicio	formulario vacío
Evento saliente	ES1	formularioResultados	formulario y las cinco operaciones
Evento saliente	ES2	formularioConErrores	lo escrito y un aviso junto a cada número no válido
Evento saliente	ES3	formularioResultadosSinDivision	suma, resta y producto, y el aviso "el segundo número es 0" (**nuevo**)
Estado	ST0	Arranque / inicio	la página aún no se ha mostrado
Estado	ST1	Atención de peticiones	el formulario está a la vista
Variable de estado	VE0	`$estado`	estado actual (campo oculto)
Variable	VE1	`$suma`	
Variable	VE2	`$resta`	
Variable	VE3	`$producto`	
Variable	VE4	`$numero1`	primer número
Variable	VE5	`$numero2`	segundo número
Variable	VE6	`$resto`	(en el original se usaba sin estar en la tabla)
Variable	VE7	`$cociente`	(**nuevo**)
Predicado	PL0	`$numero1` válido	entero entre −PHP_INT_MAX y PHP_INT_MAX
Predicado	PL1	`$numero2` válido	entero entre −PHP_INT_MAX y PHP_INT_MAX
Predicado	PL2	`$numero2` no es 0	`$numero2 !== 0` (**nuevo**; solo tiene sentido si PL1)
Acción	AC0	`$suma = $numero1 + $numero2`	calcular suma
Acción	AC1	`$resta = $numero1 - $numero2`	calcular resta
Acción	AC2	`$producto = $numero1 * $numero2`	calcular producto
Acción	AC3	`$resto = $numero1 % $numero2`	calcular resto (necesita PL2)

Tipo	Abreviatura	Nombre	Significado
Acción	AC4	`$cociente = intdiv($numero1, $numero2)`	calcular cociente (**nuevo**; necesita PL2)
Acción	AC5	validar `$numero1`	`trim` y `FILTER_VALIDATE_INT` con rango (**nuevo**: antes estaba fuera del autómata)
Acción	AC6	validar `$numero2`	ídem
Acción	AC7	avisos	un aviso junto a cada número no válido (**nuevo**)

4.2.- Tabla de eventos-estados ampliada

Igual que en el original:

	Enviar (EE0)	Reset (EE1)
ST0	T1	T1
ST1	T2	T1

4.3.- Tabla de transiciones

#	AEs antes de los PLs	PLs	Nuevo ST	AEs después	ESs
T1	---	---	ST1	---	ES0
T2 (a)	AC5, AC6	`!PL0 || !PL1`	ST1	AC7	ES2
T2 (b)	AC5, AC6	`PL0 && PL1 && !PL2`	ST1	AC0, AC1, AC2	ES3
T2 (c)	AC5, AC6	`PL0 && PL1 && PL2`	ST1	AC0, AC1, AC2, AC3, AC4	ES1

4.4.- Comprobación de predicados (T2)

La tabla del original tenía 4 filas (PL0, PL1) y dos caminos. Con PL2 hay 3 predicados, pero PL2 no influye cuando el camino (a) ya se cumple (se escribe **x**):

PL0	PL1	PL2	(a) `!PL0 || !PL1`	(b) `PL0 && PL1 && !PL2`	(c) `PL0 && PL1 && PL2`
0	0	x	1	0	0
0	1	x	1	0	0
1	0	x	1	0	0
1	1	0	0	1	0
1	1	1	0	0	1

Debes conocer

Con los dos predicados del original, la fila `1 1` lleva al camino de los resultados **también cuando el segundo número es 0**. La tabla de dos predicados era completa y excluyente, pero el camino (c) tenía una acción (AC3) que **no se puede ejecutar** en todos sus casos. Por eso, al hacer la tabla de transiciones, conviene preguntarse por cada acción: *¿qué tiene que ser cierto para que esta acción funcione?* Si la respuesta no está ya en los predicados del camino, falta un predicado.

5.- Del autómata al código

La estructura es la de la solución del material (`index.php` con `$automata` y un `switch`); las diferencias están en T2:

```
case 2:                                                  // ---- T2
    $texto1 = trim((string) ($_GET['numero1'] ?? ''));   // AC5, AC6
    $texto2 = trim((string) ($_GET['numero2'] ?? ''));
    $numero1 = enteroValido($texto1);
    $numero2 = enteroValido($texto2);
    $pl0 = $numero1 !== false;
    $pl1 = $numero2 !== false;

    if (!$pl0 || !$pl1) {                                // (a)
        $estado = ST1;
        // AC7: un aviso junto a cada número no válido
        es2($texto1, $texto2, $errores);
    } elseif ($numero2 === 0) {                          // (b): PL0 && PL1 && !PL2
        $estado = ST1;
        $suma = $numero1 + $numero2;                     // AC0
        $resta = $numero1 - $numero2;                    // AC1
        $producto = $numero1 * $numero2;                 // AC2
        es3($texto1, $texto2, $suma, $resta, $producto);
    } else {                                             // (c): PL0 && PL1 && PL2
        $estado = ST1;
        $suma = $numero1 + $numero2;                     // AC0
        $resta = $numero1 - $numero2;                    // AC1
        $producto = $numero1 * $numero2;                 // AC2
        $resto = $numero1 % $numero2;                    // AC3
        $cociente = intdiv($numero1, $numero2);          // AC4
        es1($texto1, $texto2, $suma, $resta, $producto, $cociente, $resto);
    }
    break;
```

```
function enteroValido(string $texto): int|false                // AC5 y AC6
{
    return filter_var($texto, FILTER_VALIDATE_INT, ['options' => ['min_range' => -PHP_INT_MAX,
'max_range' => PHP_INT_MAX]]);
}
```

Recomendación

¿Por qué no la expresión regular `/^[+-]?\d+$/` de `formatoEnteroValido()`? Porque comprueba el **formato** pero no si el número **cabe** en un `int`: `99999999999999999999` pasa y PHP lo convierte en `9223372036854775807` sin avisar (los resultados salen mal). Además, `intdiv(PHP_INT_MIN, -1)` lanza `ArithmeticError`. `FILTER_VALIDATE_INT` rechaza lo que no cabe, y el rango deja fuera `PHP_INT_MIN`. Por la misma razón, la suma, la resta y el producto se declaran `int|float`: con números muy grandes PHP los convierte en `float` (por ejemplo, `PHP_INT_MAX - (-1)` da `9.2233720368548E+18`). Otra diferencia: `FILTER_VALIDATE_INT` no admite ceros a la izquierda (`07`).

6.- Pruebas: un camino por cada fila

Prueba	Camino	Resultado comprobado
Primera carga	T1	formulario vacío; `estado` = 1
30 y 10	T2 (c)	suma 40, resta 20, producto **300**, cociente 3, resto 0
−7 y 2	T2 (c)	cociente −3, resto −1
5 y 0	T2 (b)	suma 5, resta 5, producto 0 y "el segundo número es 0"; sin `DivisionByZeroError`
vacío y 3	T2 (a)	aviso en el primer número; el 3 se conserva
3.5 y "abc"	T2 (a)	los dos avisos
99999999999999999999 y 1	T2 (a)	no cabe en un `int`: aviso
−9223372036854775808 y −1	T2 (a)	fuera del rango: aviso, sin `ArithmeticError`
9223372036854775807 y −1	T2 (c)	cociente −9223372036854775807, resto 0 (la resta sale como `float`)
Reset con datos	T1	formulario vacío
los dos botones a la vez	T1	se trata como Reset
Enviar sin `estado`	T1	formulario vacío
`<b>` como número	T2 (a)	se muestra escapado

7.- Comparación con el autómata del material

	Autómata del material	**Este autómata**
Estados y tabla de eventos-estados	ST0, ST1; T1, T2	iguales
Predicados	PL0, PL1 (PL2 en blanco)	PL0, PL1 y **PL2: el segundo número no es 0**
Caminos de T2	2: errores y resultados	3: errores, resultados **sin división** y resultados
Operaciones	suma, resta, producto, resto	y el **cociente** que pide el enunciado
Validación	`formatoEnteroValido()` fuera del autómata	AC5 y AC6, acciones previas; además, que el número quepa en un `int`
Página con errores	"Rellene ambos campos, por favor" y los campos vacíos	un aviso junto a cada campo no válido, conservando lo escrito

8.- Para practicar

- Añade el cociente **decimal** (`$numero1 / $numero2`, con dos decimales). ¿Necesita un predicado nuevo o le basta PL2?
- Haz que el campo con error conserve lo escrito pero el otro se vacíe. ¿Cambia algo en el autómata o solo en ES2?
